

considerable amount by automating the deployment process.

- All distributions, be it Azure, Amazon EKS, or DigitalOcean have a set of battle-tested host images, optimised for Docker and Kubernetes.
- Security is rarely an issue with cloud native distributions, especially when containers are involved. Microsoft Azure has one of the most advanced security technologies backing it (courtesy Microsoft) with tools like multi-factor authorisation and advanced threat analytics.
- Kubernetes distributions support an array of languages and are not heavy on the pocket either. DigitalOcean gets you an SSD based VM for just US\$ 5, with 512MB RAM, 20GB disk space and 1TB transfers.
- Kubernetes ensures that even after undesirable events, the system regularly heals itself so that it stays the same as time passes by. For example, if you create four replicas of an application, and one gets deleted, or destroyed manually, Kubernetes will bring one back so that the same number of replicas continue to run. This autonomous process is called a control loop, and is a popular controller implementation choice amongst users.
- The concept of decoupled microservice architectures was consolidated through Kubernetes. While building a microservice, do note that one microservice can be utilised by several teams for their service implementation. Groups of containers finally bring together the images developed by different teams into one deployable unit.
- The criticality of logs is apparent to anyone responsible for debugging programs. With Kubernetes, logging is automatic.

Kubernetes is a relatively novel concept. With the community around it and their skillsets still evolving, it can be potentially difficult to implement the concept of containers

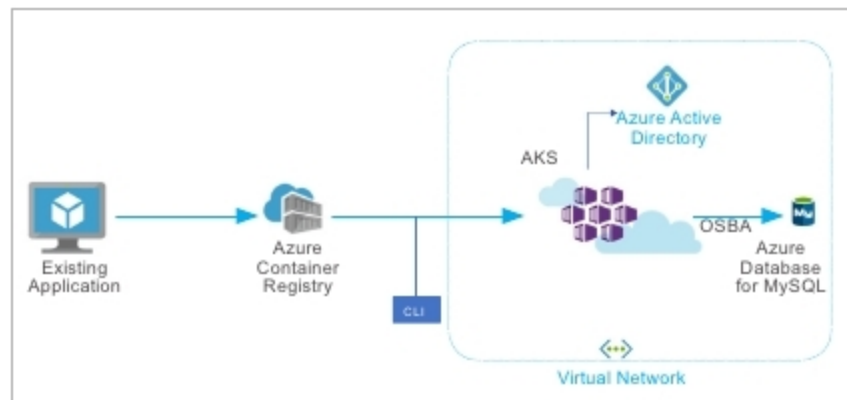


Figure 1: Shifting to containers

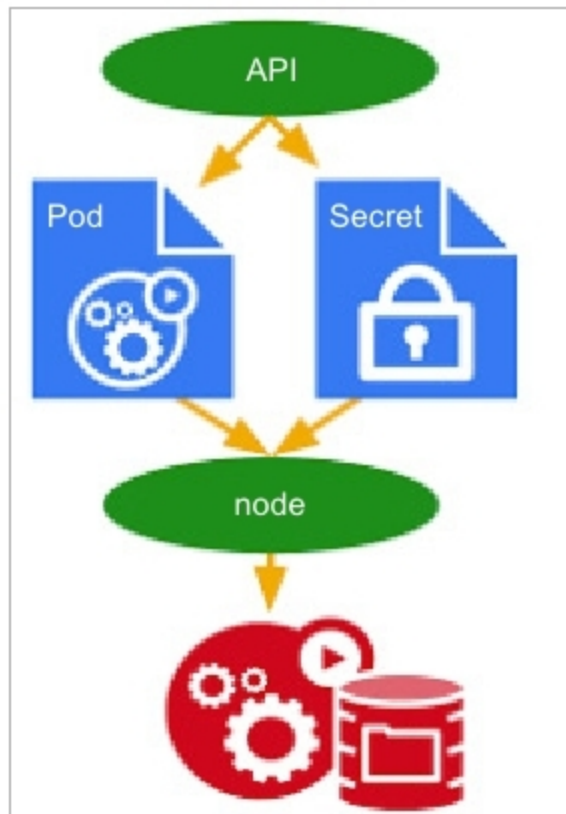


Figure 2: Kubernetes secure secrets

for existing applications. Working with Kubernetes requires a certain degree of team reorganisation and some additional employees with the desired skillsets, as it can be hard to set up and configure.

Considerations

While diving into Kubernetes, there are a number of factors an enterprise must keep in mind to avoid any roadblocks.

- *Start afresh for legacy applications:* Legacy applications are already built completely with no errors. Instead of breaking down the entire application, it is easier to either create the application from scratch, or re-architect the code base for containerisation.
- *Use secure secrets:* Kubernetes has a highly recommended secure secrets management system, which keeps API keys and tokens safe. Users only need to define the secret in the

cluster and attach the secret to the pod.

- *Set resource constraints:* It is critical to set the appropriate resource constraints. Limiting oneself to low resources comes with its obvious problems,

while taking too much may turn out to be expensive and a waste. Not specifying the limit will result in the pod using unlimited resources, which could ultimately crash the entire node.

- Kubernetes APIs may change frequently over multiple version updates due to the large community and open source base. It's advisable to refer to the current documentation for any updates and to check the latest information lest you are following tutorials that may be outdated. What might have been acceptable a couple of versions ago can be obsolete now. This happens frequently with storage, since newer ways to consume persistent storage in a K8s cluster keep popping up.

Migrating workloads to the cloud and containers can seem like a daunting task. However, with Docker and Kubernetes and their well documented tutorials, the process is certainly made a lot easier. With industry support behind both (Google designed Kubernetes and Docker has alliances with Red Hat and Microsoft), security, safety and reliability are more than reasonably assured. **END** 🐧

Reference

- [1] <https://azure.microsoft.com/en-us/blog/orchestrating-production-grade-workloads-with-azure-kubernetes-service/>

By: Rudraksh Mukta Kulshreshtha

The author is the VP-R&D at IndiQus. His work revolves around cloud native infrastructure and data engineering systems. He loves dabbling in historical research and computational art.